

# Sefi Stellar Indexing + Agent SDK

## Part 1: Protocol Indexing, Unified Semantics, and AI Agent Answering

A detailed architecture and build guide for adapting Sefi from a Hedera-oriented semantic agent layer into a Stellar-first SDK that indexes Blend, Aquarius AMM, and Stellar DEX/AMM protocol surfaces.

|             |                                                                                                                                    |
|-------------|------------------------------------------------------------------------------------------------------------------------------------|
| Scope       | Indexing + SDK + AI agent answering only. The next document should cover proof-of-data and ZK verification in depth.               |
| Constraints | Protocol-only ingestion, not full-chain indexing; \$100 AWS/month target; \$10 on-chain budget reserved for later proof contracts. |
| Date        | 2026-06-26                                                                                                                         |

Prepared for implementation by AI coding agents and human engineers.

# 1. Executive Summary

Sefi should become the agent-facing semantic SDK for selected Stellar protocols. The immediate objective is not to build a full-chain indexer and not to start with ZK. The first objective is to build a reliable protocol data plane and semantic answer layer for Blend, Aquarius AMM, and Stellar DEX/AMM. Once those are stable, proof-of-data and ZK proofs can bind agent computations to the exact raw protocol data used.

This document defines the practical indexing architecture, protocol adapters, semantic ontology, SDK surface, AI agent tools, storage schema, operational pipeline, and build phases. It is intentionally scoped as Part 1. The proof layer is designed as a first-class handoff but not implemented here.

Repo access note: I attempted to inspect <https://github.com/Kaushikh76/Sefi>, but the available GitHub fetch and clone paths failed in this environment. Therefore, this document uses the user-provided architectural definition of Sefi as a Hedera-origin blockchain semantic layer for agents and maps that concept to Stellar. When the repo is accessible to the AI agent, it should map existing modules to the migration table in Section 4.

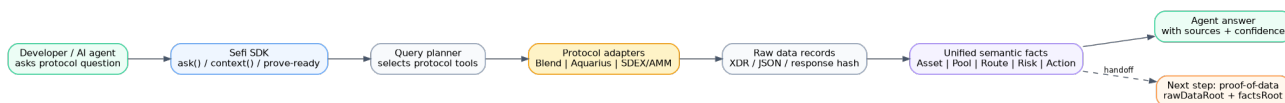


Figure 1 - Part 1 scope: protocol data ingestion, semantic normalization, SDK tools, and AI answering. The proof-of-data layer is the next build step.

The core decision is to index protocol surfaces, not the whole chain. Stellar RPC supports contract-state reads with `getLedgerEntries` and read-only contract simulation through `simulateTransaction`, while `getEvents` is explicitly intended for backend Dapp ingestion but has recent-ledger retention limits. Horizon remains necessary for Classic Stellar DEX concepts such as offers, order books, trades, paths, and liquidity pools. [S1-S3, S7-S10]

## 2. Research Basis and Feasibility Conclusions

The feasibility analysis rests on three official data surfaces and three protocol surfaces:

| Area                            | Finding                                                                        | Impact for Sefi                                                          |
|---------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Stellar RPC getLedgerEntries    | Reads current ledger entries and contract state/code                           | Use for raw ledger data; can be used to reconstruct ledger history       |
| Stellar RPC simulateTransaction | Can invoke read-only smart contract functions for free                         | Primary path for Sefi to get contract state changes, cost, latest ledger |
| Stellar RPC getEvents           | Filtered events over recent ledgers, max 7 days; default 240 events per ledger | Full 240 events per ledger; can be used to track events in the ledger    |
| Blend                           | Pool config includes name, reserves, oracle, take rate                         | Excellent for pool data; can be used to track pool data                  |
| Aquarius AMM                    | Pool data can be retrieved by smart contract calls or back-end API             | Use for pool data; can be used to track pool data                        |
| Stellar DEX/AMM                 | Order books, offers, trades, paths, and liquidity pools                        | Use for order book data; can be used to track order book data            |

Conclusion: end-to-end indexing and semantic answering is feasible under the stated budget if Sefi limits itself to three protocol surfaces and uses a hybrid on-demand + cached index design. The risky part is not data access; it is semantic correctness, stale-data handling, and preventing the LLM from answering beyond sourced facts.

Professional document tooling: this PDF uses ReportLab for structured pagination and Graphviz/DOT for repeatable architecture diagrams. Graphviz is specifically designed to render node-edge graph descriptions into diagrams, which is suitable for technical whitepaper flow charts.

### 3. Sefi Migration Model: Hedera Core to Stellar Core

Because Sefi already exists conceptually as a blockchain semantic layer for agents, the migration should preserve the product architecture and replace only chain-specific adapters. Do not rewrite Sefi as a generic Stellar dashboard. The core Sefi primitives should remain: source adapters, semantic normalization, query planning, answer generation, tool exposure, and later proof/capsule creation.

| Existing Sefi role  | Hedera-style component             | Stellar replacement                                      |
|---------------------|------------------------------------|----------------------------------------------------------|
| Network data access | Mirror Node / contract event reads | Stellar RPC, Horizon, protocol SDKs                      |
| Token model         | HTS token metadata and balances    | Stellar assets, trustlines, SAC tokens, liquidity pools  |
| Contract events     | Contract logs/events               | Soroban getEvents with protocol contract filters         |
| Current state       | Mirror/API state reads             | getLedgerEntries and simulateTransaction getters         |
| Semantic schema     | Protocol/entity/action facts       | Keep same core schema; add Stellar-specific entity types |
| Agent tools         | ask, inspect, reason, explain      | Same interface; add blend, aquarius, sdex namespaces     |
| Proof handoff       | Future hash/proof records          | RawDataRecord, ContextCapsule, factsRoot for Stellar     |

The migration priority should be adapter isolation. Existing Sefi code should be refactored until all chain-specific assumptions sit behind a `SourceAdapter` or `ProtocolAdapter` boundary. The SDK and agent tools should not care whether a fact came from Hedera Mirror Node, Stellar RPC, Horizon, or an Aquarius getter.

```
// Migration boundary: everything chain-specific lives behind this interface.
export interface SourceAdapter<TQuery, TRaw> {
  sourceName: string;
  network: 'stellar-mainnet' | 'stellar-testnet';
  fetch(query: TQuery): Promise<TRaw[]>;
  canonicalize(raw: TRaw): CanonicalRawRecord;
}

export interface ProtocolAdapter<TContext> {
  protocol: 'blend' | 'aquarius' | 'stellar_dex' | 'stellar_amm';
  buildContext(args: TContext): Promise<ContextCapsule>;
  projectFacts(capsule: ContextCapsule): Promise<SemanticFact[]>;
  answer(question: string, context: ContextCapsule): Promise<GroundedAnswer>;
}
```

# 4. Target Architecture

The target architecture has two modes: indexed mode and live context mode. Indexed mode keeps protocol events, common pool snapshots, and semantic facts available for fast agent answers. Live context mode fetches fresh state on demand for questions where stale data would be dangerous, such as borrow safety, swap execution, and route slippage.

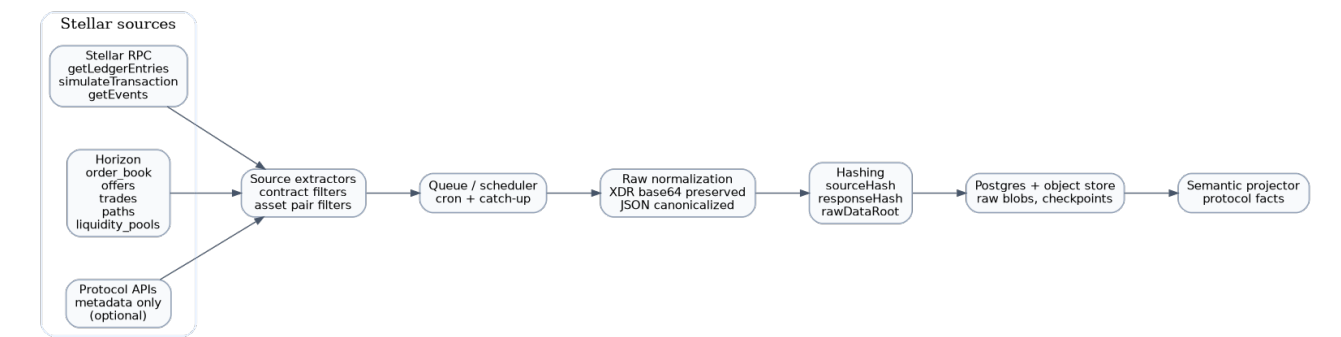


Figure 2 - Protocol-only ingestion architecture. Events and snapshots are normalized into raw records before semantic projection.

A core rule: raw source artifacts are stored before semantic projection. This makes the next proof-of-data phase possible. Every semantic fact must point back to at least one SourceRecord.

| Layer               | Responsibility                                                    | Build now? |
|---------------------|-------------------------------------------------------------------|------------|
| Source extractors   | Fetch raw XDR/JSON from RPC, Horizon, protocol contract methods.  | Yes        |
| Raw data store      | Preserve exact response, hash, ledger markers, provider metadata. | Yes        |
| Semantic projectors | Convert raw protocol records into canonical facts.                | Yes        |
| SDK tools           | Expose ask/context/compose APIs to developers and agents.         | Yes        |
| Answer engine       | Grounded natural-language answers with citations to facts.        | Yes        |
| Proof-of-data       | rawDataRoot and factsRoot are prepared but ZK binding contract.   | Repare.    |

## 5. Indexing Strategy: Protocol Surfaces, Not Full Chain

The indexer should not attempt a full Stellar history. Stellar already provides analytics/indexing infrastructure elsewhere, and the project budget is intentionally small. Sefi needs protocol-grade state for agent reasoning, not every ledger entry. Therefore the indexer tracks only contracts, assets, pairs, and pool IDs that the SDK knows about.

There are three indexing patterns:

| Pattern                   | When to use                                                | Examples                                                           |
|---------------------------|------------------------------------------------------------|--------------------------------------------------------------------|
| On-demand getter snapshot | Current state is needed and the call is cheap/cheap enough | Blend get_reserve, Aquarius get_info, Aquarius estimate_swap.      |
| Event ingestion           | State changes must be queryable beyond RPC checkpoint      | Blend position changes, Aquarius swaps/deposits/withdrawals/claims |
| Horizon market polling    | Classic DEX market state and path liquidity.               | Order books, offers, trades, paths, liquidity pools.               |

Because Stellar RPC `getEvents` has recent-ledger limits and explicitly recommends backend ingestion, Sefi needs checkpointed event jobs from day one. Each job stores `lastLedger`, `lastCursor`, and deduplicates by event id. [S2]

```
// Generic checkpoint record.
interface IngestionCheckpoint {
  id: string; // e.g. "blend:pool:C...:events"
  network: StellarNetwork;
  source: 'stellar-rpc' | 'horizon' | 'aquarius-api';
  cursor?: string;
  lastLedger?: number;
  lastClosedAt?: string;
  status: 'active' | 'paused' | 'failed';
  updatedAt: string;
}
```

## 6. Blend Indexer and Semantic Adapter

Blend should be the first adapter because it has rich lending semantics and strong SDK/read patterns. It gives Sefi a high-value agent domain: borrow safety, collateral exposure, utilization, oracle freshness, emissions, and action classification.

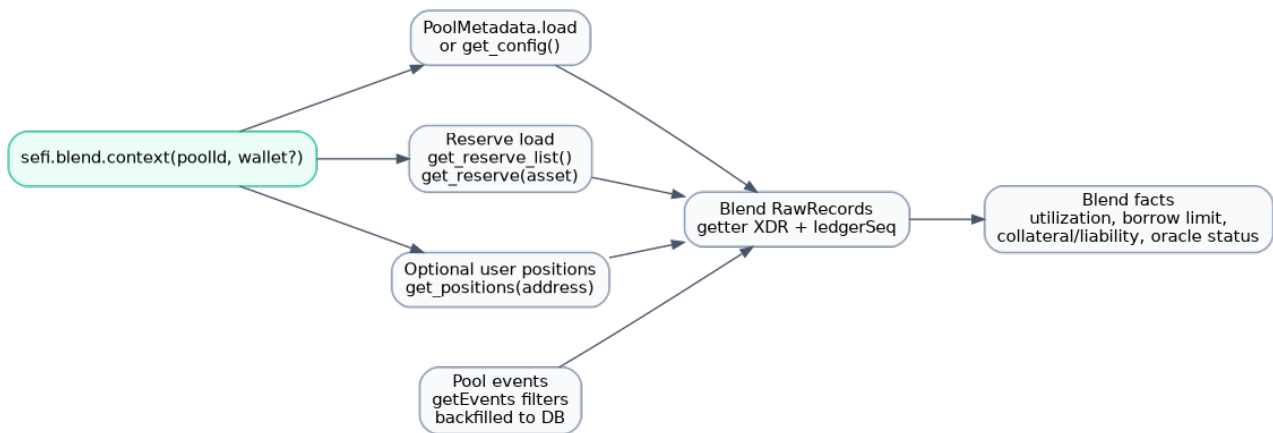


Figure 3 - Blend indexing and semantic projection.

Official Blend integration guidance says direct reads without the SDK require transaction simulation against pool contract getter functions. Pool configuration stores the pool name, supported reserves, oracle, and take rate. Reserve data includes supplied amount, borrowed amount, reserve configuration, indexes, interest rates, and emissions. User data is stored as protocol-token positions: supply, collateral, or liabilities. [S4]

### Blend ingestion jobs:

| Job                    | Source                                      | Frequency                     | Output facts                                                           |
|------------------------|---------------------------------------------|-------------------------------|------------------------------------------------------------------------|
| Pool metadata snapshot | Blend SDK PoolMetadata.load or get_config() | Once at startup + hourly      | pool.name, pool.oracle, pool.reserves, pool.takeRate                   |
| Reserve snapshot       | SDK load all reserves or get_reserve(asset) | Every 5 min for tracked pools | totalSupplied, totalBorrowed, utilization, collateralFactor, liability |
| User position snapshot | get_positions(address) for opted-in wallets | On demand                     | supply, collateral, liability, health estimate                         |
| Event catch-up         | getEvents by tracked Blend contract IDs     | Continuous polling            | positionChanged, reserveUpdated, emissionsUpdated, liquidation         |

### Blend semantic facts:

```
// Example Blend facts emitted from raw getter outputs.
{
  protocol: 'blend',
  entity: { type: 'reserve', poolId, asset: 'USDC' },
  facts: {
    totalSupplied: '1250000000000',
    totalBorrowed: '780000000000',
    utilizationBps: 6240,
    collateralFactorBps: 7500,
    liabilityFactorBps: 8500,
    oracleStatus: 'fresh',
    reserveStatus: 'active'
  },
  sourceRefs: ['raw_blend_get_reserve_USDC...']
}
```

### Agent questions Blend must answer:

- "Is this pool safe to borrow from right now?" - uses reserve utilization, oracle status, and configured risk factors.
- "What action reduces risk for this wallet?" - uses user positions and action classification.
- "What changed in this pool today?" - uses event ingestion and snapshots.
- "Can I run a private policy check later?" - prepares a ContextCapsule for proof-of-data/ZK.

## 7. Aquarius AMM Indexer and Semantic Adapter

Aquarius is the liquidity and swap semantics adapter. It complements Blend by answering whether a user can enter or exit a position, whether a route has acceptable slippage, and whether a pool is stable/volatile/reward-relevant.

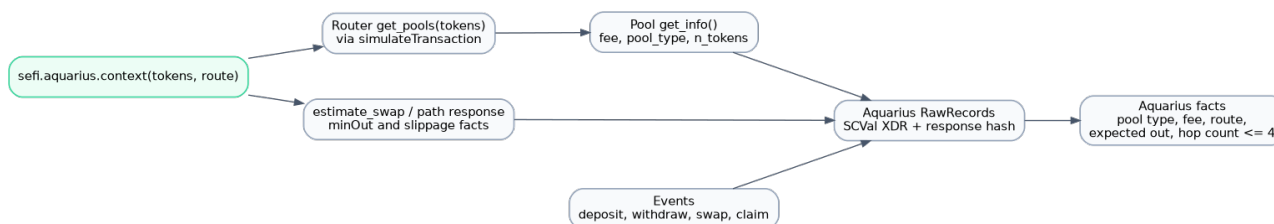


Figure 4 - Aquarius indexing and semantic projection.

Aquarius docs state that pool data can be retrieved by smart contract methods or by backend API. For Sefi, contract methods should be the proof-grade path because simulation returns raw SCVal/XDR outputs. Backend API can be used for UI metadata but should not be the canonical source for proof-of-data. [S5]

Aquarius Soroban functions include deposit, withdraw, and swap\_chained. The swap\_chained call executes a chain of swaps, takes a swap-chain vector, input token, input amount, and minimum output, and returns the output amount. Aquarius notes that up to 4 pools are currently allowed due to Soroban limitations. [S6]

| Job            | Source                                           | Frequency                       | Output facts                                    |
|----------------|--------------------------------------------------|---------------------------------|-------------------------------------------------|
| Pool discovery | router.get_pools(tokens) via simulateTransaction | On demand + daily population    | poolHash, token pair                            |
| Pool info      | pool.get_info()                                  | Every 1-5 min for tracked pools | poolType, fee, nTokens, amplification if stable |
| Swap estimate  | estimate_swap or route response                  | On demand                       | estimatedOut, slippage, routeHops, pool type    |
| Event catch-up | getEvents filtered by router/pool contract       | Continuous                      | swap, deposit, withdraw, claim events           |

```

// Aquarius context example.
const ctx = await sefi.aquarius.context({
  tokenIn: XLM_CONTRACT,
  tokenOut: USDC_CONTRACT,
  amountIn: '100000000',
  include: ['pools', 'pool_info', 'swap_estimate'],
  sourcePreference: 'contract-simulation'
});

// Semantic facts produced.
ctx.facts.route.hopCount <= 4;
ctx.facts.swap.estimatedOut;
ctx.facts.swap.slippageBps;
ctx.facts.pool.poolType; // constant_product | stable_swap
  
```

### Agent questions Aquarius must answer:

- "Can I swap this amount with less than 1% slippage?" - uses estimate\_swap and route facts.
- "Is there enough exit liquidity for my Blend debt asset?" - combines Aquarius route output with Blend amount.
- "Which route is safer: Aquarius or SDEX?" - compares normalized route facts.

## 8. Stellar DEX / Stellar AMM Indexer and Semantic Adapter

The Stellar DEX and native AMM are Classic Stellar data surfaces rather than Soroban-first protocols. They require Horizon for market abstractions and optional RPC getLedgerEntries for raw offer/liquidity-pool XDR. This adapter is essential because Stellar is still a payments and asset-routing network, not just a Soroban app chain.

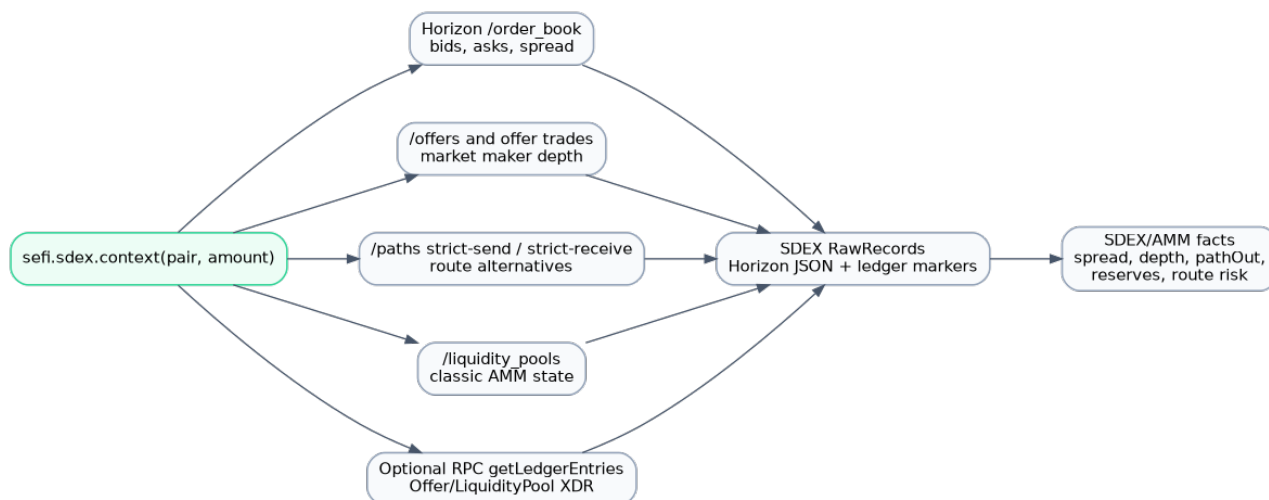


Figure 5 - Stellar DEX and native AMM indexing.

Horizon exposes order books for asset pairs, offers, trades, paths, and liquidity pool resources. getLedgerEntries can read Offer and Liquidity Pool ledger entries directly as XDR, which is useful for stronger source records. [S1, S7-S10]

| Job                | Source                                                       | Frequency                  | Output facts                                   |
|--------------------|--------------------------------------------------------------|----------------------------|------------------------------------------------|
| Orderbook snapshot | Horizon /order_book                                          | 30s-2min for tracked pairs | bestBid, bestAsk, spreadBps, depth bands       |
| Offer snapshot     | Horizon /offers and optional RPC Offer XDR                   | 1-5 min                    | maker offer depth, price levels, raw offer IDs |
| Trade ingestion    | Horizon /trades stream/poll                                  | Continuous or periodic     | recent volume, trade price, market activity    |
| Path quotes        | Horizon paths strict-send/strict-receive                     | on demand                  | pathOut, route assets, route risk              |
| Liquidity pools    | Horizon /liquidity_pools and optional RPC LiquidityPool keys | on demand                  | reserves, shares, pool liquidity               |

The SDEX adapter should treat Horizon JSON as convenient but not sufficient for future proof-of-data. Where possible, it should derive or store corresponding LedgerKey Offer and LedgerKey LiquidityPool XDR via getLedgerEntries. For path results, store the raw Horizon response and latest known ledger markers from the response metadata.

```

// SDEX semantic facts.
{
  protocol: 'stellar_dex',
  entity: { type: 'market', base: 'XLM', counter: 'USDC' },
  facts: {
    bestBid: '0.0891',
    bestAsk: '0.0894',
    spreadBps: 34,
    depthAt50Bps: '25000 USDC',
    pathAvailable: true,
    pathOut: '988.4 USDC'
  },
  sourceRefs: ['horizon_orderbook_xlm_usdc_...', 'rpc_offer_xdr_...']
}

```

## 9. Unified Semantic Ontology

The semantic layer is what makes Sefi more than three separate protocol SDKs. Every protocol adapter must project data into shared entities and fact types, so an agent can compose Blend health with Aquarius route liquidity and SDEX market depth.

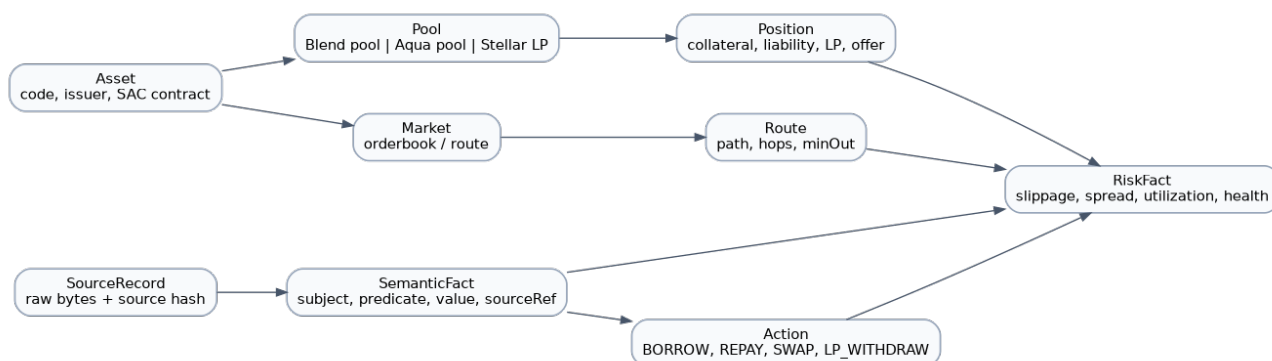


Figure 6 - Unified ontology for multi-protocol Stellar agent reasoning.

| Semantic entity | Blend                                                                 | Aquarius                                   | SDEX/AMM                                |
|-----------------|-----------------------------------------------------------------------|--------------------------------------------|-----------------------------------------|
| Asset           | Reserve asset, collateral asset, liability asset                      | Base asset, input/output token             | Base/counter asset, path asset          |
| Pool            | Lending pool                                                          | AMM pool                                   | Native liquidity pool                   |
| Market          | Oracle market only                                                    | Swap route                                 | Orderbook market/path                   |
| Position        | Collateral, liability, supply                                         | LP shares                                  | Offer, LP share, path exposure          |
| RiskFact        | Health, borrow limit, utilization, or supply                          | Slippage, route hops, pool type            | Spread, depth, pathOut                  |
| Action          | SUPPLY, BORROW, REPAY, WITHDRAW, LP_DEPOSIT, LP_WITHDRAW, LP_WITHDRAW | SWAP, LP_DEPOSIT, LP_WITHDRAW, LP_WITHDRAW | PLACE_OFFER, CANCEL_OFFER, PATH_PAYMENT |

The ontology should be versioned. Every fact emitted by an adapter should carry adapterHash, schemaVersion, sourceRefs, ledgerSeq, and confidence. This is important for future proof-of-data, replayability, and agent safety.

```

export interface SemanticFact<T = unknown> {
  factId: string;
  protocol: 'blend' | 'aquarius' | 'stellar_dex' | 'stellar_amm';
  subject: { type: string; id: string; asset?: AssetRef; poolId?: string };
  predicate: string; // e.g. 'risk.utilizationBps'
  value: T;
  unit?: string;
  ledgerSeq?: number;
  ledgerClosedAt?: string;
  sourceRefs: string[];
  adapterHash: string;
  schemaVersion: string;
  confidence: 'direct' | 'derived' | 'estimated' | 'api_metadata';
}

```

# 10. Raw Source Records and Proof-of-Data Handoff

Although this document is not the full proof-of-data implementation, the indexer must be built as if proof-of-data is coming next. That means every raw response must be retained or reproducibly addressable. No semantic fact should be orphaned from its raw source.

MVP terminology:

| Term           | Meaning now                                            | Future proof-layer meaning                       |
|----------------|--------------------------------------------------------|--------------------------------------------------|
| SourceRecord   | Exact raw response or XDR used by the adapter          | Leaf in rawDataRoot.                             |
| ContextCapsule | Bundle of source records, semantic facts, and metadata | Proof input to proof of data/computation.        |
| rawDataRoot    | Hash/Merkle root of raw source records.                | Data binding for ZK proof.                       |
| factsRoot      | Hash/Merkle root of semantic facts.                    | Semantic binding for proof and verifier.         |
| adapterHash    | Hash of adapter code/config version.                   | Proves which parser/semantic projector was used. |

```
export interface SourceRecord {
  sourceId: string;
  network: StellarNetwork;
  protocol: 'blend' | 'aquarius' | 'stellar_dex' | 'stellar_amm';
  provider: 'stellar-rpc' | 'horizon' | 'protocol-api';
  method: string; // getLedgerEntries, simulateTransaction, /order_book, etc.
  requestHash: string;
  responseHash: string;
  rawFormat: 'xdr-base64' | 'json-canonical' | 'mixed';
  rawPointer: string; // S3/object-store path or DB blob id
  latestLedger?: number;
  ledgerSeq?: number;
  ledgerClosedAt?: string;
  fetchedAt: string;
  adapterHash: string;
}

export interface ContextCapsule {
  capsuleId: string;
  protocols: string[];
  sourceRecordIds: string[];
  rawDataRoot: string;
  factsRoot: string;
  adapterSetHash: string;
  createdAt: string;
  freshness: { maxAgeSeconds: number; oldestLedger?: number; newestLedger?: number };
}
```

Part 2 should implement root construction, replay scripts, multi-RPC quorum, and on-chain/verifier binding. Part 1 only guarantees that the raw data is captured and every answer can point back to the exact data used.

# 11. Database and Storage Design

The cheapest reliable architecture for the first month is Postgres for metadata and semantic facts, object storage for raw blobs, and a small vector index for protocol docs plus semantic fact summaries. SQLite is possible for local demo, but Postgres is preferable for concurrent workers and agent queries.

```
-- Core tables
CREATE TABLE source_records (
  source_id TEXT PRIMARY KEY,
  network TEXT NOT NULL,
  protocol TEXT NOT NULL,
  provider TEXT NOT NULL,
  method TEXT NOT NULL,
  request_hash TEXT NOT NULL,
  response_hash TEXT NOT NULL,
  raw_format TEXT NOT NULL,
  raw_pointer TEXT NOT NULL,
  latest_ledger BIGINT,
  ledger_seq BIGINT,
  ledger_closed_at TIMESTAMPTZ,
  fetched_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  adapter_hash TEXT NOT NULL
);

CREATE TABLE semantic_facts (
  fact_id TEXT PRIMARY KEY,
  protocol TEXT NOT NULL,
  subject_type TEXT NOT NULL,
  subject_id TEXT NOT NULL,
  predicate TEXT NOT NULL,
  value_json JSONB NOT NULL,
  unit TEXT,
  ledger_seq BIGINT,
  source_refs TEXT[] NOT NULL,
  adapter_hash TEXT NOT NULL,
  schema_version TEXT NOT NULL,
  confidence TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE ingestion_checkpoints (
  id TEXT PRIMARY KEY,
  network TEXT NOT NULL,
  source TEXT NOT NULL,
  cursor TEXT,
  last_ledger BIGINT,
  status TEXT NOT NULL DEFAULT 'active',
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_facts_lookup ON semantic_facts(protocol, subject_type, subject_id, predicate);
CREATE INDEX idx_sources_protocol_ledger ON source_records(protocol, latest_ledger);
```

## Object-store layout:

```
s3://sefi-raw/stellar-mainnet/blend/{poolId}/{yyyy}/{mm}/{dd}/{sourceId}.json
s3://sefi-raw/stellar-mainnet/aquarius/{poolId}/{yyyy}/{mm}/{dd}/{sourceId}.json
s3://sefi-raw/stellar-mainnet/sdex/{assetPair}/{yyyy}/{mm}/{dd}/{sourceId}.json
s3://sefi-capsules/{capsuleId}.json
```

## 12. Sefi SDK Architecture

The SDK should feel like agent tools first and data pipelines second. Developers should not think about RPC, Horizon, XDR, or protocol-specific getter naming unless they opt into advanced mode.

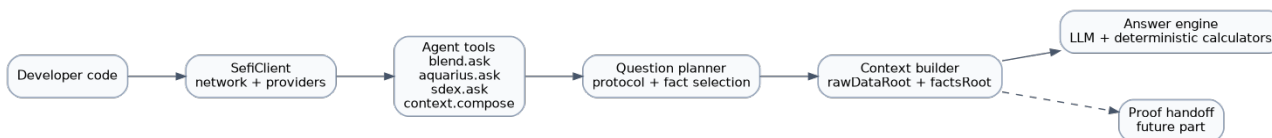


Figure 7 - SDK layers exposed to developers and agents.

### SDK namespaces:

| Namespace     | Purpose                                         | Example                                              |
|---------------|-------------------------------------------------|------------------------------------------------------|
| sefi.blend    | Blend context, facts, and Q&A.                  | sefi.blend.ask("Is this pool risky?", { poolId })    |
| sefi.aquarius | Aquarius pool, route, and liquidity context.    | sefi.aquarius.route({ tokenIn, tokenOut, amountIn }) |
| sefi.sdex     | Classic DEX and native AMM market context.      | sefi.sdex.market({ base, counter })                  |
| sefi.context  | Build single or composite ContextCapsules.      | sefi.context.compose({ blend, aquarius, sdex })      |
| sefi.agent    | Expose tools for LangChain/OpenAI-style agents. | sefi.agent.tools(["blend.ask", "context.compose"])   |

```

import { SefiClient } from '@sefi/stellar-sdk';

const sefi = new SefiClient({
  network: 'stellar-mainnet',
  rpcUrl: process.env.STELLAR_RPC_URL,
  horizonUrl: 'https://horizon.stellar.org',
  rawDataMode: 'capture', // capture raw data for proof-of-data handoff
});

const answer = await sefi.blend.ask(
  'Can this pool support a conservative USDC borrow?',
  { poolId: BLEND_POOL_ID, asset: USDC }
);

console.log(answer.summary);
console.log(answer.sources); // SourceRecord + SemanticFact refs
  
```

## 13. AI Agent Answering Layer

The agent must not directly hallucinate from raw docs or unaudited numbers. The answer loop should force every response through deterministic context tools. The LLM explains the result, but the calculators and semantic facts produce the result.



Figure 8 - Grounded AI answer loop.

### Agent answer contract:

```
interface GroundedAnswer {
  question: string;
  summary: string;
  decision?: 'safe' | 'unsafe' | 'unknown' | 'needs_fresh_context';
  confidence: 'high' | 'medium' | 'low';
  assumptions: string[];
  evidence: Array<{
    factId: string;
    sourceIds: string[];
    value: unknown;
    freshnessSeconds?: number;
  }>;
  warnings: string[];
  nextActions: Array<'refresh_context' | 'prepare_proof' | 'simulate_tx' | 'ask_user'>;
}
```

### The agent can answer three classes of questions:

| Question type   | Example                                                  | Required path                                                   |
|-----------------|----------------------------------------------------------|-----------------------------------------------------------------|
| Informational   | What is the difference between Blend and Aqua? Does RAG? | Domain-specific semantic ontology.                              |
| Data-grounded   | Is the XLM/USDC route liquid enough right now?           | Wide context + deterministic route/slippage calculator.         |
| Decision/action | Should my agent borrow and swap?                         | Composite context + policy guardrails; proof handoff if needed. |

### Guardrail rules:

- If data is stale beyond the context policy, answer must include "needs\_fresh\_context".
- If a fact is derived, show source facts and formula in the answer metadata.
- If protocols disagree or one source fails, mark confidence low and ask for refresh or fallback.
- If the user asks for execution, route through simulation/action-validity tooling, not direct LLM construction.

## 14. Multi-Protocol Context and Agent Reasoning

Multi-protocol reasoning is the differentiator. A normal indexer can answer one protocol query; Sefi should answer cross-protocol agent questions such as whether a Blend borrow has exit liquidity through Aquarius or SDEX, or whether an LP withdrawal route can repay a Blend liability.

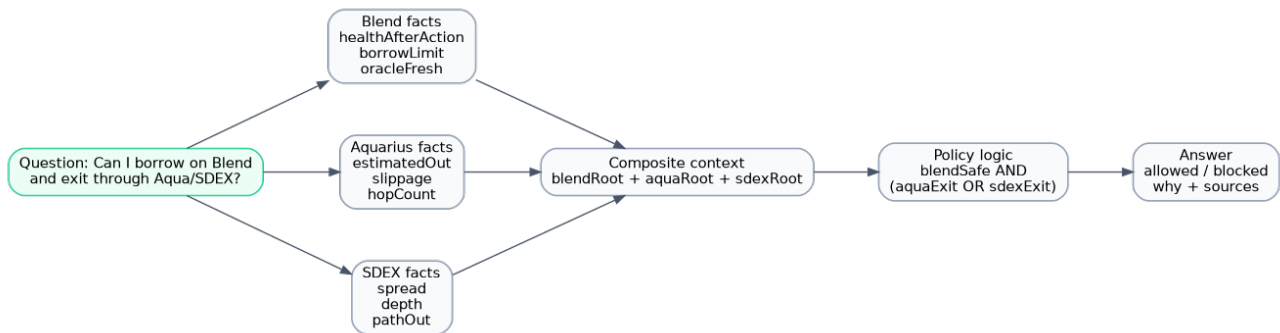


Figure 9 - Multi-protocol context composition for agent answers.

```
const composite = await sefi.context.compose({
  blend: {
    poolId: BLEND_POOL_ID,
    wallet: userWallet,
    include: ['pool_config', 'reserves', 'oracle', 'positions']
  },
  aquarius: {
    tokenIn: USDC,
    tokenOut: XLM,
    amountIn: '100000000',
    include: ['pools', 'pool_info', 'swap_estimate']
  },
  sdex: {
    base: XLM,
    counter: USDC,
    amount: '100000000',
    include: ['orderbook', 'paths', 'liquidity_pools']
  }
});

const answer = await sefi.agent.answer(
  'Can I borrow USDC from Blend and exit safely through Aquarius or SDEX?',
  { context: composite, policy: { maxSlippageBps: 100, minHealth: 1.25 } }
);
```

The agent should not expose private policy values unless the user requests it. In Part 2, those policy values become private inputs for proof workflows.

# 15. Example Use Cases

These are the first high-value experiences the SDK should support.

| Use case                 | Prompt                                                                                                       | Data required                                                           | Output                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Blend borrow safety      | User asks if a wallet can borrow mblend.USD/reserve/oracle/user position                                     | blend.USD/reserve/oracle/user position                                  | safe/unsafe with reasons and stale-data warnings.                       |
| Exit liquidity check     | Before a borrow, agent checks if the borrowed asset Aquarius route through SDEX meets slippage policy.       | blend.reserve.Aquarius route through SDEX meets slippage policy.        | blend.reserve.Aquarius route through SDEX meets slippage policy.        |
| LP withdrawal planning   | User wants to withdraw liquidity and Aquarius SDEX paths + Blend fallback path, expected output, risk notes. | Aquarius SDEX paths + Blend fallback path, expected output, risk notes. | Aquarius SDEX paths + Blend fallback path, expected output, risk notes. |
| Market route explanation | User asks why Sefi picked Aquarius SDEX estimate, SDEX path, spread, depth, etc.                             | Aquarius SDEX estimate, SDEX path, spread, depth, etc.                  | Aquarius SDEX estimate, SDEX path, spread, depth, etc.                  |
| Proof preparation        | User asks for a future verifiable claim                                                                      | ContextCapsule + SourceRecords.                                         | rawDataRoot/factsRoot ready for Part 2.                                 |

## Use-case sample answer:

User: Can my agent borrow 100 USDC from Blend and still exit safely?

Sefi answer:

- Blend status: Safe under current context. Borrow action keeps health estimate above policy.
- Aquarius exit: Acceptable. Route has estimated output above the minimum and hop count <= 4.
- SDEX fallback: Available but worse. Spread is higher and depth is lower.
- Decision: Allowed, but refresh context before execution if older than 30 seconds.
- Evidence: blend.reserve.USDC.utilization, blend.user.healthAfterAction, aquarius.route.estimatedOut, sdex.market.spreadBps.

## 16. Indexer Implementation Details

The indexer should be written as small workers, not one monolith. Each worker has a source adapter, a checkpoint, and an output table. For hackathon/demo speed, use TypeScript throughout because Stellar JS SDK, Blend SDK, Aquarius examples, and agent SDK development are all JS/TS-friendly.

| Worker                | Inputs                                                       | Outputs                           | Failure handling                                 |
|-----------------------|--------------------------------------------------------------|-----------------------------------|--------------------------------------------------|
| blend-snapshot-worker | tracked pool IDs, assets, optional <del>source</del> records | source_records, semantic_facts    | retry with backoff, mark stale if fetch fails    |
| blend-event-worker    | contract IDs, topics, checkpoint                             | event source_records, event facts | dedupe event id, resume cursor                   |
| aquarius-pool-worker  | token pairs, router contract                                 | pool records, pool facts          | fallback to API metadata if getter unavailable   |
| aquarius-route-worker | route requests, amount bands                                 | swap estimate facts               | on-demand only for execution-sensitive questions |
| sdex-market-worker    | asset pairs                                                  | orderbook/trade/path facts        | Horizon rate-limit backoff, stale flags          |

### Recommended repository structure:

```
packages/  
  sefi-core/  
    src/ontology/  
    src/source-records/  
    src/context-capsule/  
    src/hash/  
  sefi-stellar/  
    src/rpc/  
    src/horizon/  
    src/assets/  
  sefi-protocol-blend/  
    src/adapter.ts  
    src/projector.ts  
    src/calculators.ts  
  sefi-protocol-aquarius/  
    src/adapter.ts  
    src/projector.ts  
    src/route.ts  
  sefi-protocol-sdex/  
    src/adapter.ts  
    src/orderbook.ts  
    src/paths.ts  
  sefi-agent-sdk/  
    src/client.ts  
    src/tools.ts  
    src/answer.ts  
apps/  
  api-server/  
  indexer-worker/  
  demo-agent-ui/
```

## 17. Documentation RAG and Semantic Memory

AI answering needs both live facts and stable protocol knowledge. Stable knowledge should come from curated docs embeddings; live facts should come from context capsules. The agent should never answer current-state questions from documentation alone.

| Knowledge type   | Source                                | Storage               | Use                                        |
|------------------|---------------------------------------|-----------------------|--------------------------------------------|
| Protocol docs    | Blend docs, Aquarius docs, Stellar RP | Curated docs          | Explain concepts and choose tools.         |
| Live facts       | SourceRecords projected into Semantic | Reliable              | Answer current-state questions.            |
| Derived formulas | Sefi calculators and formulas         | Versioned source code | Deterministic risk and route calculations. |
| Answer history   | Past questions and contexts           | Postgres optional     | Debug and improve planning.                |

### A robust answer pipeline:

1. Classify question:
  - concept explanation
  - current-state analysis
  - multi-protocol decision
  - action/proof preparation
2. Retrieve docs snippets only for concepts/tool selection.
3. Build live ContextCapsule for current-state questions.
4. Run deterministic calculators.
5. Generate answer with evidence metadata.
6. Apply stale-data and missing-source guardrails.
7. Return answer + sources + next recommended action.

# 18. Deployment Architecture Under \$100 AWS Credits

A \$100/month budget is enough for a small demo deployment if the indexer is protocol-scoped and avoids high-frequency full-market polling. The most cost-effective deployment is one small VM or container service, managed Postgres if credits allow, and object storage for raw blobs.

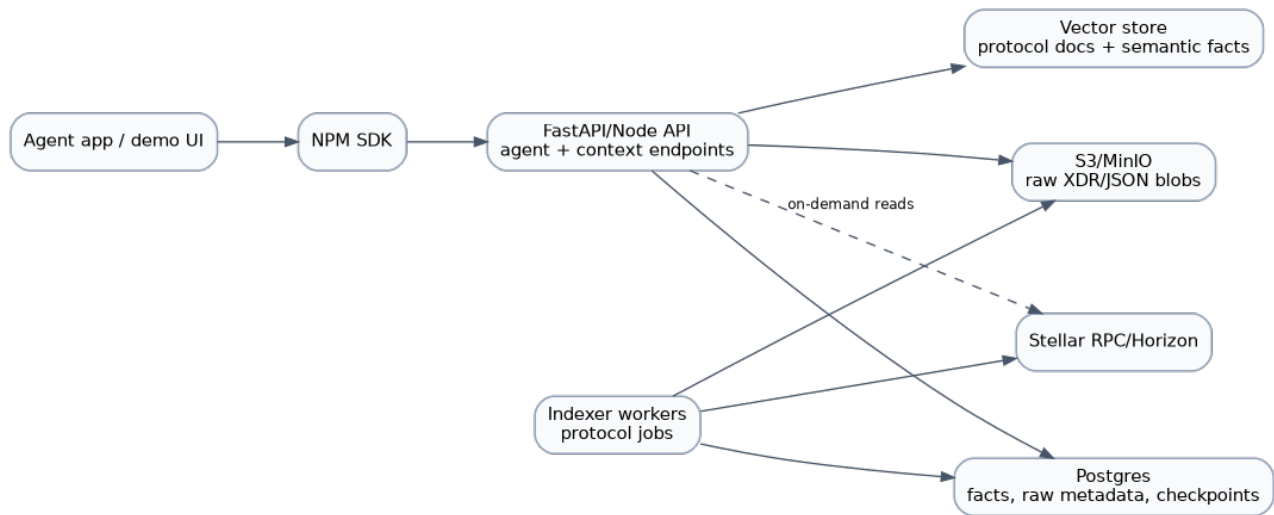


Figure 10 - Low-cost deployment architecture.

| Component          | Low-cost choice                              | Notes                                      |
|--------------------|----------------------------------------------|--------------------------------------------|
| API + agent server | t3.small/t4g.small EC2 or single ECS service | Run Node API, SDK backend, and agent demo. |
| Workers            | Same VM with PM2/systemd or small ECS tasks  | Starts with scheduled jobs, not Kafka.     |
| Database           | RDS Postgres small or self-hosted Postgres   | RDS is easier; self-hosted is cheaper.     |
| Object storage     | S3                                           | Raw XDR/JSON blobs are cheap.              |
| Vector store       | pgvector or local SQLite/Chroma              | Keep it small; docs only.                  |
| RPC/Horizon        | Public providers at first                    | Add provider rotation and caching.         |

## Polling defaults to stay cheap:

|                       |                                                        |
|-----------------------|--------------------------------------------------------|
| Blend tracked pools:  | snapshot every 60-300 seconds                          |
| Blend user positions: | on demand only                                         |
| Aquarius pool info:   | every 60-300 seconds for tracked pairs                 |
| Aquarius estimates:   | on demand only                                         |
| SDEX orderbooks:      | every 30-120 seconds for tracked pairs                 |
| SDEX paths:           | on demand only                                         |
| Events:               | polling windows with checkpoint; low interval for demo |
| Raw blobs:            | gzip JSON/XDR records before storing                   |

# 19. Detailed Build Phases

The build should be phased so every stage produces a demoable product. Do not wait for proof-of-data before building the semantic answer layer.

| Phase                    | Deliverable                                                                                        | Acceptance criteria                                                            |
|--------------------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| 0 - Repo migration audit | Map existing Sefi modules to SourceAdapter, ProtocolAdapter, Agent SDK, and Stellar adapter stubs. | Adapter stubs complete.                                                        |
| 1 - Stellar source layer | RPC and Horizon clients with canonical SourceRecord creation.                                      | Can fetch getLedgerEntries, simulateTransaction, getEvents, Horizon orderbook. |
| 2 - Blend adapter        | Blend context + semantic facts + answer tool.                                                      | Can answer pool utilization/risk question with raw source refs.                |
| 3 - Aquarius adapter     | Pool discovery, get_info, swap estimate, route facts.                                              | Can answer slippage/route question.                                            |
| 4 - SDEX/AMM adapter     | Orderbook, path, offers, liquidity pool facts.                                                     | Can answer spread/depth/path question.                                         |
| 5 - Composite context    | Blend + Aquarius + SDEX context.compose.                                                           | Can answer borrow + exit liquidity question.                                   |
| 6 - Agent SDK            | Agent tools and answer contract.                                                                   | LangChain/OpenAI-style agent can use tools safely.                             |
| 7 - Proof handoff        | rawDataRoot, factsRoot, capsule export.                                                            | Part 2 can consume capsules for proof-of-data/ZK.                              |

## Phase 1 implementation checklist:

- [ ] Create @sefi/stellar-source package.
- [ ] Implement RpcClient wrapper for getLedgerEntries, simulateTransaction, getEvents.
- [ ] Implement HorizonClient wrapper for order\_book, offers, trades, paths, liquidity\_pools.
- [ ] Implement canonical JSON serializer and hash function.
- [ ] Implement SourceRecord writer.
- [ ] Implement ingestion\_checkpoint table.
- [ ] Add raw blob compression and object-store writer.
- [ ] Add tests with saved fixtures for every source method.

## 20. Code Context for the AI Coding Agent

The AI coding agent should start from interfaces and fixtures. The first PR should not include LLM integration. It should include deterministic source adapters and tests.

### Context builder skeleton:

```
class StellarContextBuilder {
  constructor(
    private rpc: StellarRpcClient,
    private horizon: HorizonClient,
    private rawStore: RawStore,
    private factStore: FactStore,
  ) {}

  async blendPool(args: BlendContextArgs): Promise<ContextCapsule> {
    const raw = await this.fetchBlendRaw(args);
    const sourceRecords = await this.rawStore.writeMany(raw);
    const facts = BlendProjector.project(sourceRecords);
    await this.factStore.writeMany(facts);
    return ContextCapsule.from({ sourceRecords, facts });
  }

  async compose(parts: ContextCapsule[]): Promise<ContextCapsule> {
    return ContextCapsule.compose(parts);
  }
}
```

### Agent tool skeleton:

```
export function makeSefiAgentTools(sefi: SefiClient) {
  return [
    tool('blend_ask', async ({ question, poolId, wallet }) => {
      return sefi.blend.ask(question, { poolId, wallet });
    }),
    tool('aquarius_route_check', async ({ tokenIn, tokenOut, amountIn }) => {
      const ctx = await sefi.aquarius.context({ tokenIn, tokenOut, amountIn });
      return sefi.aquarius.answer('Check route quality', { context: ctx });
    }),
    tool('sdex_market_check', async ({ base, counter, amount }) => {
      const ctx = await sefi.sdex.context({ base, counter, amount });
      return sefi.sdex.answer('Check market depth', { context: ctx });
    }),
    tool('compose_context', async (args) => sefi.context.compose(args)),
  ];
}
```

## 21. Correctness, Freshness, and Safety Rules

Agent answers are only useful if users know when to trust them. Sefi must publish freshness windows, source quality, and confidence levels in every answer. This is also essential for later proof-of-data.

| Risk                             | Mitigation                                                                                           |
|----------------------------------|------------------------------------------------------------------------------------------------------|
| RPC stale or inconsistent data   | Store latestLedger and fetchedAt; optionally multi-provider quorum for important contexts.           |
| Horizon response not proof-grade | Use Horizon for market convenience; use getLedgerEntries for Offer/LiquidityPool XDR when available. |
| LLM hallucination                | Use LLM only after deterministic facts and calculations. Require evidence array.                     |
| Event gaps                       | Checkpoint getEvents; respect 7-day window; monitor missed-window errors.                            |
| Protocol adapter bug             | Version adapterHash; add fixtures; store raw and projected facts for replay.                         |
| Over-indexing cost               | Track only configured pools/pairs and use on-demand estimates.                                       |

### Freshness policy example:

```
const freshnessPolicy = {
  blend: {
    poolSnapshotMaxAgeSec: 120,
    userPositionMaxAgeSec: 30,
    oracleStatusMaxAgeSec: 60,
  },
  aquarius: {
    poolInfoMaxAgeSec: 300,
    swapEstimateMaxAgeSec: 20,
  },
  sdex: {
    orderbookMaxAgeSec: 30,
    pathQuoteMaxAgeSec: 20,
  }
};
```

## 22. Handoff to Part 2: Proof of Data on Stellar

This document intentionally stops before implementing ZK. However, the data architecture is proof-ready. The next step should take ContextCapsules and bind them to Stellar-verifiable proofs.

Part 2 should answer:

- How to construct rawDataRoot and factsRoot deterministically.
- Whether to use Noir, RISC Zero, or another backend for each proof recipe.
- How to deploy a Soroban verifier registry under the \$10 on-chain budget.
- How to strengthen raw-data origin using multi-RPC quorum, replay scripts, or ledger inclusion/state proof research.
- How to prove multi-protocol computes: Blend health AND (Aquarius route OR SDEX path).

The key design goal remains: Part 1 makes answers grounded; Part 2 makes selected computations verifiable.

# 23. References

Primary sources used for the architecture and feasibility analysis:

| ID  | Source                                                                                                                                        | URL                                                                                                                                                                                                       |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S1  | Stellar RPC getLedgerEntries: live ledger state, contract state/code, XDR entries, stateful ledger, and more info/docs/getLedgerEntries       | <a href="https://developers.stellar.org/docs/data/apis/rpc/api-reference/methods/getLedgerEntries">https://developers.stellar.org/docs/data/apis/rpc/api-reference/methods/getLedgerEntries</a>           |
| S2  | Stellar RPC getEvents: filtered contract events, 7-day maximum query window, 20MB max response size, pagination, and more info/docs/getEvents | <a href="https://developers.stellar.org/docs/data/apis/rpc/api-reference/methods/getEvents">https://developers.stellar.org/docs/data/apis/rpc/api-reference/methods/getEvents</a>                         |
| S3  | Stellar RPC simulateTransaction: trial contract invocation, read-only smart contract, and more info/docs/simulateTransaction                  | <a href="https://developers.stellar.org/docs/data/apis/rpc/api-reference/methods/simulateTransaction">https://developers.stellar.org/docs/data/apis/rpc/api-reference/methods/simulateTransaction</a>     |
| S4  | Blend integration docs: pool config, reserves, user positions, getter simulation, and more info/docs/integrations/integrate-pool              | <a href="https://blendprotocol.com/docs/integrations/integrate-pool">https://blendprotocol.com/docs/integrations/integrate-pool</a>                                                                       |
| S5  | Aquarius Get Pools Info docs: pool data by smart contract methods or backend API, and more info/docs/get-pools-info                           | <a href="https://aqua.network/docs/get-pools-info">https://aqua.network/docs/get-pools-info</a>                                                                                                           |
| S6  | Aquarius Soroban functions: deposit, withdraw, swap_chained, swap-chain, and more info/docs/soroban-functions                                 | <a href="https://aqua.network/docs/soroban-functions">https://aqua.network/docs/soroban-functions</a>                                                                                                     |
| S7  | Stellar Horizon order books: order-book aggregation for asset pairs.                                                                          | <a href="https://developers.stellar.org/docs/data/apis/horizon/api-reference/aggregations/order-books">https://developers.stellar.org/docs/data/apis/horizon/api-reference/aggregations/order-books</a>   |
| S8  | Stellar Horizon offers: offers resource and offer trade access.                                                                               | <a href="https://developers.stellar.org/docs/data/apis/horizon/api-reference/resources/offers">https://developers.stellar.org/docs/data/apis/horizon/api-reference/resources/offers</a>                   |
| S9  | Stellar Horizon liquidity pools: liquidity pool resource and related effects, trade, and more info/docs/liquidity-pools                       | <a href="https://developers.stellar.org/docs/data/apis/horizon/api-reference/resources/liquidity-pools">https://developers.stellar.org/docs/data/apis/horizon/api-reference/resources/liquidity-pools</a> |
| S10 | Stellar Horizon paths: strict-send and strict-receive path finding for path payments, and more info/docs/paths                                | <a href="https://developers.stellar.org/docs/data/apis/horizon/api-reference/aggregations/paths">https://developers.stellar.org/docs/data/apis/horizon/api-reference/aggregations/paths</a>               |
| S11 | Stellar ZK docs: ZK primitives and proof verification direction on Stellar; next steps, and more info/docs/build/apps/zk                      | <a href="https://developers.stellar.org/docs/build/apps/zk">https://developers.stellar.org/docs/build/apps/zk</a>                                                                                         |

End of Part 1. Recommended next document: Sefi Proof-of-Data + ZK Verification on Stellar.